

TP1 Analyse en Composantes Principales

Ghazi Bel Mufti

26 mars 2020

La base de données {Decathlon} se trouve dans le package FactoMineR. Les données contiennent les performances d'athlètes lors de deux compétitions. Le tableau de données contient 41 lignes et 13 colonnes. Les colonnes de 1 à 12 sont des variables continues: les dix premières colonnes correspondent aux performances des athlètes pour les dix épreuves du décathlon et les colonnes 11 et 12 correspondent respectivement au rang et au nombre de points obtenus. La dernière colonne est une variable qualitative correspondant au nom de la compétition (Jeux Olympiques de 2004 ou Décastar 2004).

Objectifs de l'ACP

L'ACP permet de décrire un jeu de données, de le résumer, d'en réduire la dimensionnalité. L'ACP réalisée sur les individus du tableau de données répond à différentes questions :

- Etude des individus (i.e. des athlètes) : deux athlètes sont proches s'ils ont des résultats similaires. On s'intéresse à la variabilité entre individus. Y a-t-il des similarités entre les individus pour toutes les variables ? Peut-on établir des profils d'athlètes ? Peut-on opposer un groupe d'individus à un autre ?
- Etude des variables (i.e. des performances) : on étudie les liaisons linéaires entre les variables. Les objectifs sont de résumer la matrice des corrélations et de chercher des variables synthétiques: peut-on résumer les performances des athlètes par un petit nombre de variables ?
- Lien entre les deux études : peut-on caractériser des groupes d'individus par des variables ?

Importer et explorer le jeu de données

```
library(FactoMineR)
data(decathlon)
head(decathlon)
```

##	100m	Long.jump	Shot.put	High.jump	400m	110m.hurdle	Discus
## SEBRLE	11.04	7.58	14.83	2.07	49.81	14.69	43.75
## CLAY	10.76	7.40	14.26	1.86	49.37	14.05	50.72
## KARPOV	11.02	7.30	14.77	2.04	48.37	14.09	48.95
## BERNARD	11.02	7.23	14.25	1.92	48.93	14.99	40.87
## YURKOV	11.34	7.09	15.19	2.10	50.42	15.31	46.26

```
## WARNERS 11.11      7.60      14.31      1.98 48.68      14.23  41.10
##      Pole.vault Javeline 1500m Rank Points Competition
## SEBRLE      5.02      63.19 291.7      1      8217      Decastar
## CLAY      4.92      60.15 301.5      2      8122      Decastar
## KARPOV      4.92      50.31 300.2      3      8099      Decastar
## BERNARD      5.32      62.77 280.1      4      8067      Decastar
## YURKOV      4.72      63.44 276.4      5      8036      Decastar
## WARNERS      4.92      51.77 278.1      6      8030      Decastar
```

```
tail(decathlon)
```

```
##      100m Long.jump Shot.put High.jump 400m 110m.hurdle
## Turi      11.08      6.91      13.62      2.03 51.67      14.26
## Lorenzo    11.10      7.03      13.22      1.85 49.34      15.38
## Karlivans   11.33      7.26      13.30      1.97 50.54      14.98
## Korkizoglou 10.86      7.07      14.81      1.94 51.16      14.96
## Uldal      11.23      6.99      13.53      1.85 50.95      15.09
## Casarsa    11.36      6.68      14.92      1.94 53.20      15.39
##      Discus Pole.vault Javeline 1500m Rank Points Competition
## Turi      39.83      4.8      59.34 290.01      23      7708      OlympicG
## Lorenzo    40.22      4.5      58.36 263.08      24      7592      OlympicG
## Karlivans   43.34      4.5      52.92 278.67      25      7583      OlympicG
## Korkizoglou 46.07      4.7      53.05 317.00      26      7573      OlympicG
## Uldal      43.01      4.5      60.00 281.70      27      7495      OlympicG
## Casarsa    48.66      4.4      58.62 296.12      28      7404      OlympicG
```

```
str(decathlon) # competition variable nominale, catégorielle comme sexe (H ou F). Le reste des variable
```

```
## 'data.frame': 41 obs. of 13 variables:
## $ 100m : num 11 10.8 11 11 11.3 ...
## $ Long.jump : num 7.58 7.4 7.3 7.23 7.09 7.6 7.3 7.31 6.81 7.56 ...
## $ Shot.put : num 14.8 14.3 14.8 14.2 15.2 ...
## $ High.jump : num 2.07 1.86 2.04 1.92 2.1 1.98 2.01 2.13 1.95 1.86 ...
## $ 400m : num 49.8 49.4 48.4 48.9 50.4 ...
## $ 110m.hurdle: num 14.7 14.1 14.1 15 15.3 ...
## $ Discus : num 43.8 50.7 49 40.9 46.3 ...
## $ Pole.vault : num 5.02 4.92 4.92 5.32 4.72 4.92 4.42 4.42 4.92 4.82 ...
## $ Javeline : num 63.2 60.1 50.3 62.8 63.4 ...
## $ 1500m : num 292 302 300 280 276 ...
## $ Rank : int 1 2 3 4 5 6 7 8 9 10 ...
## $ Points : int 8217 8122 8099 8067 8036 8030 8004 7995 7802 7733 ...
## $ Competition: Factor w/ 2 levels "Decastar","OlympicG": 1 1 1 1 1 1 1 1 1 1 ...
```

```
summary(decathlon)
```

```
##      100m      Long.jump      Shot.put      High.jump
## Min. :10.44 Min. :6.61 Min. :12.68 Min. :1.850
## 1st Qu.:10.85 1st Qu.:7.03 1st Qu.:13.88 1st Qu.:1.920
## Median :10.98 Median :7.30 Median :14.57 Median :1.950
## Mean :11.00 Mean :7.26 Mean :14.48 Mean :1.977
## 3rd Qu.:11.14 3rd Qu.:7.48 3rd Qu.:14.97 3rd Qu.:2.040
## Max. :11.64 Max. :7.96 Max. :16.36 Max. :2.150
```

```
##      400m      110m.hurdle      Discus      Pole.vault
## Min.   :46.81 Min.   :13.97 Min.   :37.92 Min.   :4.200
## 1st Qu.:48.93 1st Qu.:14.21 1st Qu.:41.90 1st Qu.:4.500
## Median :49.40 Median :14.48 Median :44.41 Median :4.800
## Mean   :49.62 Mean   :14.61 Mean   :44.33 Mean   :4.762
## 3rd Qu.:50.30 3rd Qu.:14.98 3rd Qu.:46.07 3rd Qu.:4.920
## Max.   :53.20 Max.   :15.67 Max.   :51.65 Max.   :5.400
##      Javeline      1500m      Rank      Points
## Min.   :50.31 Min.   :262.1 Min.   : 1.00 Min.   :7313
## 1st Qu.:55.27 1st Qu.:271.0 1st Qu.: 6.00 1st Qu.:7802
## Median :58.36 Median :278.1 Median :11.00 Median :8021
## Mean   :58.32 Mean   :279.0 Mean   :12.12 Mean   :8005
## 3rd Qu.:60.89 3rd Qu.:285.1 3rd Qu.:18.00 3rd Qu.:8122
## Max.   :70.52 Max.   :317.0 Max.   :28.00 Max.   :8893
##      Competition
## Decastar:13
## OlympicG:28
##
##
##
##
```

ACP normée pas à pas

On commence par définir la matrice X composée des scores des dix épreuves du decathlon.

On définit la matrice des poids $D = \frac{1}{n}I_n$, n étant la taille de l'échantillon ainsi que la métrique $M = I_p$.

1. Calcul de la matrice centrée réduite

Centrer et réduire la matrice X

```
X=as.matrix(decathlon[,1:10])
g=colMeans(X)
g
```

```
##      100m Long.jump Shot.put High.jump      400m
## 10.998049  7.260000 14.477073  1.976829 49.616341
## 110m.hurdle Discus Pole.vault Javeline      1500m
## 14.605854 44.325610  4.762439 58.316585 279.024878
```

```
Y=sweep(x = X,2,g,FUN = '-')
round(colMeans(X),3)
```

```
##      100m Long.jump Shot.put High.jump      400m
## 10.998  7.260  14.477  1.977 49.616
## 110m.hurdle Discus Pole.vault Javeline      1500m
## 14.606 44.326  4.762 58.317 279.025
```

Calcul des écarts-types pour réduire les variables.

```
n=nrow(X)
p=ncol(X)
et=apply(Y,2,function(x) sqrt(sum(x^2)/n))
et
```

```
##          100m   Long.jump   Shot.put   High.jump         400m
##  0.25979560  0.31251927  0.81431175  0.08785906  1.13929751
## 110m.hurdle   Discus   Pole.vault   Javeline         1500m
##  0.46599998  3.33639725  0.27458865  4.76759315 11.53001177
```

Calcul de la matrice des données entrées réduites Z . On vérifie que les variables de cette matrice sont bien de variance égale à 1.

```
Z=sweep(x = Y,2,et,FUN = '/')
colSums(Z^2)/n
```

```
##          100m   Long.jump   Shot.put   High.jump         400m
##           1         1         1         1         1
## 110m.hurdle   Discus   Pole.vault   Javeline         1500m
##           1         1         1         1         1
```

2. Calcul de la matrice des corrélations

Calcul de la matrice des corrélations $R = Z'DZ$, ses valeurs propres et ses vecteurs propres.

```
M=diag(rep(1,p))
D=(1/n)*diag(rep(1,n))
R=t(Z)%*%D%*%Z
vp=eigen(R %*%M)
lambda=vp$values
lambda
```

```
## [1] 3.2719055 1.7371310 1.4049167 1.0568504 0.6847735 0.5992687
## [7] 0.4512353 0.3968766 0.2148149 0.1822275
```

```
U=vp$vectors
U
```

```
##          [,1]      [,2]      [,3]      [,4]      [,5]
## [1,]  0.42829627  0.1419891  0.15557953  0.03678703 -0.36518741
## [2,] -0.41015201 -0.2620794 -0.15372674 -0.09901016 -0.04432336
## [3,] -0.34414444  0.4539470  0.01972378 -0.18539458 -0.13431954
## [4,] -0.31619436  0.2657761  0.21894349  0.13189684 -0.67121760
## [5,]  0.37571570  0.4320460 -0.11091758 -0.02850297  0.10597034
## [6,]  0.41255442  0.1735910  0.07815576 -0.28290068 -0.19857266
## [7,] -0.30542571  0.4600244 -0.03623770  0.25259074  0.12667770
## [8,] -0.02783081 -0.1368411 -0.58361717 -0.53649480 -0.39873734
## [9,] -0.15319802  0.2405071  0.32874217 -0.69285498  0.36873120
## [10,] 0.03210733  0.3598049 -0.65987362  0.15669648  0.18557094
```

```
##           [,6]      [,7]      [,8]      [,9]      [,10]
## [1,]  0.29607739 -0.38177608  0.46160211  0.10475771  0.42428269
## [2,] -0.30612478 -0.62769317 -0.02101165  0.48266910  0.08104448
## [3,]  0.30547299  0.30972542 -0.31393005  0.42729075  0.39028424
## [4,] -0.46777116  0.09145002  0.12509166 -0.24366054 -0.10642724
## [5,] -0.33252178  0.12442114  0.21339819  0.55212939 -0.41399532
## [6,] -0.09963776 -0.35733030 -0.71111429 -0.15013429 -0.09086448
## [7,]  0.44937288 -0.42988982  0.03838986 -0.15480715 -0.44916580
## [8,]  0.26166458  0.09796019  0.17803824 -0.08297769 -0.27645138
## [9,] -0.16320268 -0.10674519  0.29614206 -0.24732691  0.08777340
## [10,] -0.29826888 -0.08362898  0.01371744 -0.30773397  0.42923132
```

Vérifions que les vecteurs propres (i.e. les colonnes de U) sont bien orthornormés.

```
round(t(U)%*%U,3)
```

```
##           [,1] [,2] [,3] [,4] [,5] [,6] [,7] [,8] [,9] [,10]
## [1,]      1    0    0    0    0    0    0    0    0    0
## [2,]      0    1    0    0    0    0    0    0    0    0
## [3,]      0    0    1    0    0    0    0    0    0    0
## [4,]      0    0    0    1    0    0    0    0    0    0
## [5,]      0    0    0    0    1    0    0    0    0    0
## [6,]      0    0    0    0    0    1    0    0    0    0
## [7,]      0    0    0    0    0    0    1    0    0    0
## [8,]      0    0    0    0    0    0    0    1    0    0
## [9,]      0    0    0    0    0    0    0    0    1    0
## [10,]     0    0    0    0    0    0    0    0    0    1
```

3. Les composantes principales

Calculons la matrice Psi des composantes principales qui est donnée par $Psi = Zu$. On vérifiera que la variance de chaque composante est égale à la valeur propre correspondante.

```
Psi=Z%*%U
Psi
```

```
##           [,1]      [,2]      [,3]      [,4]
## SEBRLE    -0.791627717  0.77161120 -0.8268411940 -1.17462736
## CLAY      -1.234990563  0.57457807 -2.1412469664  0.35484483
## KARPOV    -1.358214936  0.48402090 -1.9562579869  1.85652411
## BERNARD     0.609515083 -0.87462853 -0.8899406619 -2.22061245
## YURKOV     0.585968338  2.13095422  1.2251567968 -0.87357915
## WARNERS   -0.356889530 -1.68495667 -0.7665531449  0.58930466
## ZSIVOCZKY -0.271774781 -1.09377558  1.2827673831  1.62156457
## McMULLEN  -0.587516189  0.23072991  0.4176329823  1.52423275
## MARTINEAU  1.995359298  0.56099598  0.7299466011  0.54219127
## HERNU      1.546076462  0.48838301 -0.8407858519 -0.33119470
## BARRAS     1.341652727 -0.31091157  0.0003683375  0.64525336
## NOOL       2.344973806 -1.96637500  1.3364815492 -0.19530310
## BOURGUIGNON 3.979041865  0.19986019 -1.3264851034 -0.52435051
## Sebrle    -4.038448501  1.36582606  0.2899565043 -1.94113411
## Clay      -3.919365157  0.83696136 -0.2311753205 -1.49397212
```

## Karpov	-4.619987275	0.03999523	0.0415857980	1.31352566
## Macey	-2.233460566	1.04176620	1.8643620154	0.74321353
## Warners	-2.168396445	-1.80320025	-0.8510173287	0.28459958
## Zsivoczky	-0.925132183	1.16865180	1.4774802908	-0.80759472
## Hernu	-0.889037852	-0.61842522	0.8982953480	0.13478508
## Nool	-0.295305667	-1.54561667	-1.3552601286	-2.19972249
## Bernard	-1.906334368	-0.08580429	0.7571859709	1.45097918
## Schwarzl	-0.081078659	-1.35345710	-0.8224866222	-0.39909143
## Pogorelov	-0.539677028	0.77075099	-1.3476197769	0.55215692
## Schoenbeck	-0.114430985	-0.03985061	-0.7404039810	-0.92884762
## Barras	-0.002145203	0.36033768	1.5696934888	-0.61238304
## Smith	-0.870310570	1.05932552	1.6434290616	1.12132654
## Averyanov	-0.349155138	-1.55864999	-0.2825354037	0.02726334
## Ojaniemi	-0.380113999	-0.77244734	0.3709431419	-0.68727919
## Smirnov	0.484514213	-1.06066118	1.2283378499	-0.56603194
## Qi	0.434466691	-0.32614690	1.0697978123	0.20497404
## Drews	0.248684024	-3.08167683	-1.0548427375	0.64577513
## Parkhomenko	1.069429104	2.09318218	0.9999839029	-1.53455272
## Terek	0.681953059	0.53561440	-2.2091259997	-0.10862305
## Gomez	0.289889208	-1.19671611	1.3061025895	-0.07785116
## Turi	1.541813056	0.42716773	-0.5140859441	0.14284738
## Lorenzo	2.408509980	-1.58292969	1.5023461069	-0.30103076
## Karlivans	1.994368727	-0.29418240	0.3427836937	1.27206999
## Korkizoglou	0.957829813	2.06638554	-2.5865525263	1.19146757
## Uldal	2.562259591	0.24546871	0.4191406445	0.02118842
## Casarsa	2.857088268	3.79784505	-0.0305611909	0.73769370
##	[,5]	[,6]	[,7]	[,8]
## SEBRLE	-0.70715903	-1.03062025	-0.55152286	0.43565550
## CLAY	1.97457138	0.69012566	-0.70797408	0.60341904
## KARPOV	-0.79521472	0.73275122	-0.18993920	0.25029693
## BERNARD	-0.36163619	0.27559819	0.04961070	-0.06745808
## YURKOV	-1.25136918	-0.10460569	-0.57392548	-0.09460361
## WARNERS	-1.00166155	0.03235612	-0.09659035	0.30044536
## ZSIVOCZKY	-0.04407327	0.18537032	-0.54300336	0.73915738
## McMULLEN	-0.25151965	-1.76788298	0.10495329	0.25748521
## MARTINEAU	-1.57821348	2.36187721	-0.33238326	0.44812186
## HERNU	0.23498029	0.22249804	-1.56637865	0.06731710
## BARRAS	-0.31628748	0.40938985	0.31646694	0.65609217
## NOOL	-0.83022767	-0.99433721	-0.87390607	-0.07083076
## BOURGUIGNON	-0.29001247	-0.04908478	-0.18826373	-0.52289350
## Sebrle	-0.37695454	0.06778623	-0.55497694	0.75259621
## Clay	1.03760852	-0.81264979	-0.86751544	0.30284530
## Karpov	-0.18772953	0.74161145	-0.45414303	-1.07084207
## Macey	-0.97727010	-0.04001698	-0.19270849	-0.68974257
## Warners	0.15139464	-0.07938750	0.06100253	-0.21454500
## Zsivoczky	-0.87297257	-0.25856515	0.31352231	-0.54933741
## Hernu	-0.63498488	0.64075332	0.56048799	0.31778062
## Nool	0.03538887	0.40536270	-0.19880131	-0.30034913
## Bernard	-0.34164564	-1.08624555	0.46768687	-0.33134950
## Schwarzl	-0.28836991	0.08523813	0.08615387	0.71657047
## Pogorelov	-0.97229498	-0.36345544	0.82134446	0.47857370
## Schoenbeck	0.85442565	0.48675267	0.37063763	0.42065982
## Barras	0.67506540	0.90389490	0.44240244	0.64577089
## Smith	2.01990789	1.29179510	0.92942803	0.10317674

## Averyanov	0.36098809	-0.61650428	1.35164082	-0.76357457
## Ojaniemi	0.49925676	-0.79491103	0.22760918	-1.62981377
## Smirnov	0.40718384	0.15711173	0.36720227	-0.26809288
## Qi	0.53559673	-0.42721799	-0.94756780	0.17805603
## Drews	0.17841420	0.18998231	0.46206001	0.54928962
## Parkhomenko	-0.28180687	-0.02343641	1.72466895	0.21356688
## Terek	-1.04315650	1.23637214	0.68973066	-1.32531512
## Gomez	1.26143770	0.49836092	0.14287075	-0.09667920
## Turi	0.03871969	-1.58272797	1.29143851	1.53696070
## Lorenzo	0.68195932	0.06822736	-0.37175217	-0.96612514
## Karlivans	-0.37322060	-0.56539057	-0.97187597	0.11879004
## Korkizoglou	0.80089104	-0.80314710	0.16523591	-0.96695318
## Uldal	1.25954121	-0.19808228	-0.49760283	0.04826482
## Casarsa	0.77044963	-0.08494663	-0.26532308	-0.21238691
##	[,9]	[,10]		
## SEBRLE	-0.137558873	0.500773776		
## CLAY	-0.649244121	-0.266119255		
## KARPOV	-0.800653566	0.523268830		
## BERNARD	-0.723281017	0.188459291		
## YURKOV	-0.202216418	0.056442514		
## WARNERS	0.607465094	0.721284785		
## ZSIVOCZKY	-0.354412930	-0.146059166		
## McMULLEN	-0.538115021	-0.329648746		
## MARTINEAU	0.399108572	-0.584484592		
## HERNU	1.322902163	0.224961868		
## BARRAS	-0.280275720	0.787396307		
## NOOL	-0.507299066	0.224544692		
## BOURGUIGNON	-0.418431784	0.019430261		
## Sebrle	0.062225128	0.633130750		
## Clay	-0.013214514	-0.818729281		
## Karpov	-0.180314690	0.124574958		
## Macey	0.438424818	-0.166834121		
## Warners	0.167391548	0.082692117		
## Zsivoczky	-0.450901316	-0.307612539		
## Hernu	-0.100140875	-0.301487915		
## Nool	-0.133023057	-0.365393514		
## Bernard	0.213024073	-0.052371717		
## Schwarzl	0.624647042	-0.456745967		
## Pogorelov	0.788644760	-0.262352584		
## Schoenbeck	0.657717817	-0.299437858		
## Barras	-0.033610560	0.300976456		
## Smith	-0.191201887	0.089439874		
## Averyanov	0.751397746	-0.487116486		
## Ojaniemi	0.505551532	0.517965031		
## Smirnov	-0.455029343	-0.540450528		
## Qi	-0.274429259	-0.396922190		
## Drews	-0.075890729	-0.192719322		
## Parkhomenko	-0.115141938	0.395896413		
## Terek	-0.389166469	-0.358281849		
## Gomez	0.399815735	1.252146708		
## Turi	-0.147061945	-0.115731380		
## Lorenzo	-0.312209844	-0.168379968		
## Karlivans	0.276702842	-0.138001002		
## Korkizoglou	-0.240688329	0.444241723		

```
## Uldal      0.003274377  0.001399909
## Casarsa    0.505220025 -0.334146282
```

```
round(t(Psi)%*D%*Psi,3)
```

```
##      [,1] [,2] [,3] [,4] [,5] [,6] [,7] [,8] [,9] [,10]
## [1,] 3.272 0.000 0.000 0.000 0.000 0.000 0.000 0.000 0.000 0.000
## [2,] 0.000 1.737 0.000 0.000 0.000 0.000 0.000 0.000 0.000 0.000
## [3,] 0.000 0.000 1.405 0.000 0.000 0.000 0.000 0.000 0.000 0.000
## [4,] 0.000 0.000 0.000 1.057 0.000 0.000 0.000 0.000 0.000 0.000
## [5,] 0.000 0.000 0.000 0.000 0.685 0.000 0.000 0.000 0.000 0.000
## [6,] 0.000 0.000 0.000 0.000 0.000 0.599 0.000 0.000 0.000 0.000
## [7,] 0.000 0.000 0.000 0.000 0.000 0.000 0.451 0.000 0.000 0.000
## [8,] 0.000 0.000 0.000 0.000 0.000 0.000 0.000 0.397 0.000 0.000
## [9,] 0.000 0.000 0.000 0.000 0.000 0.000 0.000 0.000 0.215 0.000
## [10,] 0.000 0.000 0.000 0.000 0.000 0.000 0.000 0.000 0.000 0.182
```

4. Les coordonnées des variables sur les axes

Calculons la matrice Eta des coordonnées des variables sur les axes principaux par $Eta_{\alpha} = \sqrt{(\lambda_{\alpha})}u_{\alpha}$.

```
Eta<-sweep(U,2,sqrt(lambda),FUN='*')
Eta
```

```
##      [,1]      [,2]      [,3]      [,4]      [,5]
## [1,] 0.77471983 0.1871420 0.18440714 0.03781826 -0.30219639
## [2,] -0.74189974 -0.3454213 -0.18221105 -0.10178564 -0.03667805
## [3,] -0.62250255 0.5983033 0.02337844 -0.19059161 -0.11115082
## [4,] -0.57194530 0.3502936 0.25951193 0.13559420 -0.55543957
## [5,] 0.67960994 0.5694378 -0.13146970 -0.02930198 0.08769157
## [6,] 0.74624532 0.2287933 0.09263738 -0.29083103 -0.16432095
## [7,] -0.55246652 0.6063134 -0.04295225 0.25967143 0.10482712
## [8,] -0.05034151 -0.1803569 -0.69175665 -0.55153397 -0.32995932
## [9,] -0.27711085 0.3169891 0.38965541 -0.71227728 0.30512892
## [10,] 0.05807706 0.4742238 -0.78214280 0.16108904 0.15356189
##      [,6]      [,7]      [,8]      [,9]      [,10]
## [1,] 0.22920075 -0.25645445 0.290800753 0.04855323 0.18111827
## [2,] -0.23697868 -0.42164691 -0.013236949 0.22370807 0.03459636
## [3,] 0.23647411 0.20805510 -0.197770097 0.19804125 0.16660497
## [4,] -0.36211310 0.06143068 0.078805424 -0.11293209 -0.04543178
## [5,] -0.25741324 0.08357871 0.134436894 0.25590161 -0.17672678
## [6,] -0.07713202 -0.24003322 -0.447988786 -0.06958442 -0.03878833
## [7,] 0.34787054 -0.28877439 0.024184898 -0.07175021 -0.19174040
## [8,] 0.20256095 0.06580383 0.112160780 -0.03845860 -0.11801187
## [9,] -0.12633919 -0.07170506 0.186563995 -0.11463138 0.03746881
## [10,] -0.23089724 -0.05617697 0.008641731 -0.14262892 0.18323074
```

ACP normée avec le package FactoMineR et interprétation de l'ACP.

Utilisons maintenant la fonction PCA pour retrouver les résultats obtenus précédemment.

- On va ajouter deux types de variables comme variables supplémentaires : on ajoute les variables "Rank" and "Points" comme variables continues illustratives et la variable "compétition" comme variable qualitative illustrative. Les variables illustratives n'influencent pas la construction des composantes principales de l'analyse.
- Notons que nous utilisons le package factoextra plutôt que FactoMineR pour la qualité de ces graphiques.

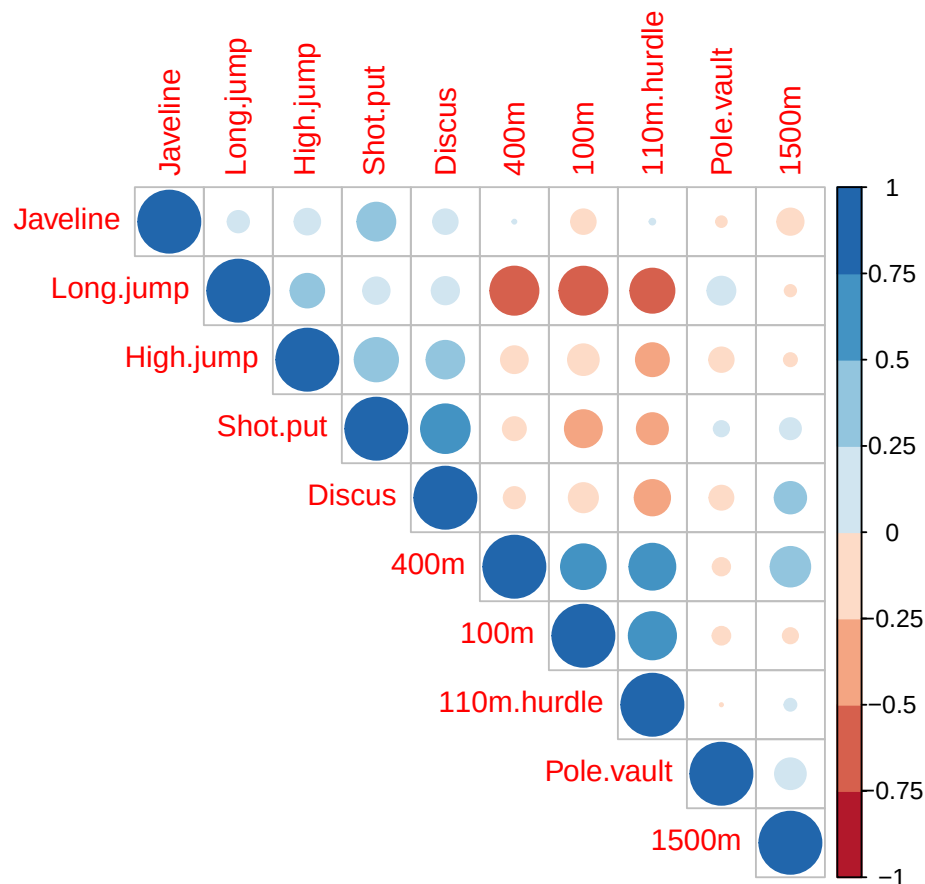
1. Pertinence de l'ACP

Le corrgram donné ci-dessous permet d'étudier les corrélations entre les variables quantitatives : il est clair qu'il existe des corrélations importantes entre des groupes de variables ce qui suggère la pertinence de cette ACP (par exemple, 100m, 400m et 110m.hurdle).

```
library(corrplot)
```

```
## corrplot 0.92 loaded
```

```
X=as.matrix(decathlon[,c(1:10)])
M<-cor(X)
library(RColorBrewer)
corrplot(M, type="upper", order="hclust",
         col=brewer.pal(n=8, name="RdBu"))
```



Execution de la fonction PCA.

```
library(factoextra)
```

```
## Loading required package: ggplot2
```

```
## Welcome! Want to learn more? See two factoextra-related books at https://goo.gl/ve3WBa
```

```
res.pca=PCA(decathlon,ncp = 5,quanti.sup = 11:12,quali.sup = 13,graph = F)
```

2. Choix du nombre d'axes à retenir

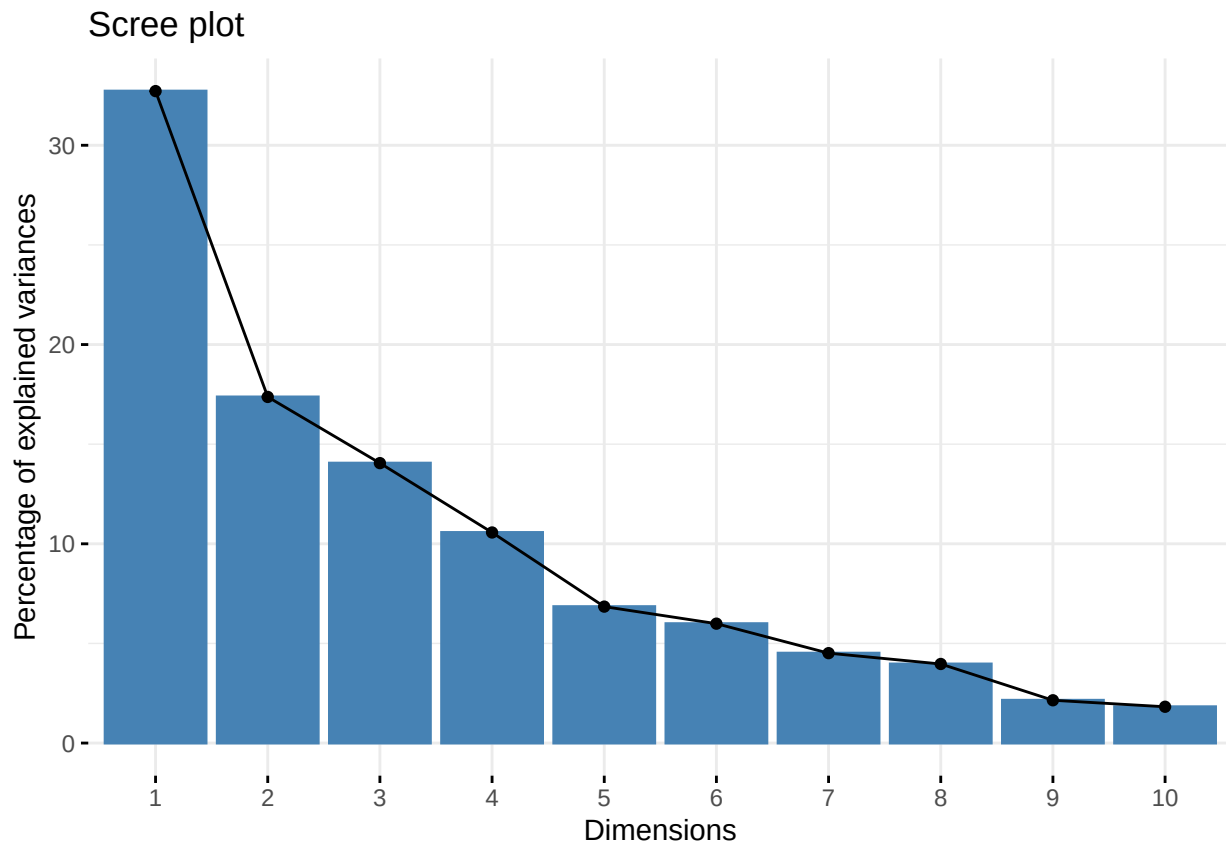
Trois critères devront être utilisés : taux d'inertie cumulé, critère de Kaiser et critère du coude.

L'objet eig est une matrice à trois colonnes contenant respectivement les valeurs propres de l'ACP, la proportion de variance de chaque composante et les variance cumulées par les composantes principales.

```
head(res.pca$eig)
```

```
##      eigenvalue percentage of variance
## comp 1  3.2719055             32.719055
## comp 2  1.7371310             17.371310
## comp 3  1.4049167             14.049167
## comp 4  1.0568504             10.568504
## comp 5  0.6847735              6.847735
## comp 6  0.5992687              5.992687
##      cumulative percentage of variance
## comp 1             32.71906
## comp 2             50.09037
## comp 3             64.13953
## comp 4             74.70804
## comp 5             81.55577
## comp 6             87.54846
```

```
fviz_screepLOT(res.pca, ncp=10)
```



- a) Critère de kaiser : on remarque qu'il y a 4 axes dont les valeurs propres sont supérieures à 1 donc on retient 4 axes d'après ce critère.
- b) Critère du taux d'inertie cumulée : On remarque que le taux d'inertie cumulé des 2 premiers axes est de 50.09% qui est un taux important compte tenu du fait que nous avons 10 variables : on va donc, d'après ce critère, retenir les 2 premiers axes. (Remarquons que retenir 3 axes pour un taux d'inertie de 64.13% est envisageable aussi).
- c) Critère du coude : On remarque que le coude se trouve au niveau du deuxième axe (voir la figure Scree plot), d'après ce critère, on devrait retenir les 2 premiers axes.

En faisant une sorte de vote des 3 critères on devrait retenir les 2 premiers axes.

3. Interprétation de la carte des variables

L'objet `var` de `res.pca` contient les 4 objets : `coord`, `cor`, `cos2` et `contrib`. A noter que vu que notre ACP est normée, `cor` (i.e. la corrélations d'une variable avec la composante principale d'un axe) est identique à `coord` (i.e. la coordonnée de cette variable sur cet axe).

```
names(res.pca$var)
```

```
## [1] "coord" "cor" "cos2" "contrib"
```

L'objet coord dans var contient les coordonnées des variables.

```
res.pca$var$coord
```

##	Dim.1	Dim.2	Dim.3	Dim.4	Dim.5
## 100m	-0.77471983	0.1871420	-0.18440714	-0.03781826	0.30219639
## Long.jump	0.74189974	-0.3454213	0.18221105	0.10178564	0.03667805
## Shot.put	0.62250255	0.5983033	-0.02337844	0.19059161	0.11115082
## High.jump	0.57194530	0.3502936	-0.25951193	-0.13559420	0.55543957
## 400m	-0.67960994	0.5694378	0.13146970	0.02930198	-0.08769157
## 110m.hurdle	-0.74624532	0.2287933	-0.09263738	0.29083103	0.16432095
## Discus	0.55246652	0.6063134	0.04295225	-0.25967143	-0.10482712
## Pole.vault	0.05034151	-0.1803569	0.69175665	0.55153397	0.32995932
## Javeline	0.27711085	0.3169891	-0.38965541	0.71227728	-0.30512892
## 1500m	-0.05807706	0.4742238	0.78214280	-0.16108904	-0.15356189

L'objet cos2 dans var est une matrice dont les lignes représentent le cos carrés de la variable (soit le carrée des coordonnées puisque l'ACP est normée).

```
res.pca$var$cos2
```

##	Dim.1	Dim.2	Dim.3	Dim.4
## 100m	0.600190812	0.03502213	0.0340059930	0.0014302206
## Long.jump	0.550415232	0.11931587	0.0332008675	0.0103603165
## Shot.put	0.387509426	0.35796686	0.0005465513	0.0363251605
## High.jump	0.327121422	0.12270561	0.0673464410	0.0183857880
## 400m	0.461869674	0.32425938	0.0172842817	0.0008586058
## 110m.hurdle	0.556882084	0.05234639	0.0085816841	0.0845826853
## Discus	0.305219255	0.36761593	0.0018448960	0.0674292539
## Pole.vault	0.002534268	0.03252860	0.4785272696	0.3041897208
## Javeline	0.076790421	0.10048206	0.1518313365	0.5073389244
## 1500m	0.003372945	0.22488818	0.6117473613	0.0259496775
##	Dim.5			
## 100m	0.091322660			
## Long.jump	0.001345279			
## Shot.put	0.012354505			
## High.jump	0.308513117			
## 400m	0.007689811			
## 110m.hurdle	0.027001375			
## Discus	0.010988725			
## Pole.vault	0.108873151			
## Javeline	0.093103658			
## 1500m	0.023581254			

Interprétation de cette première carte des variables (i.e. axes 1 et 2) :

Les deux premières dimensions contiennent 50% de l'inertie totale (l'inertie est la variance totale du tableau de données, i.e. la trace de la matrice des corrélations).

La variable "X100m" est négativement corrélée à la variable "long.jump". Quand un athlète réalise un temps faible au 100m, il peut sauter loin. Il faut faire attention ici qu'une petite valeur pour les variables "X100m", "X400m", "X110m.hurdle" et "X1500m" correspond à un score élevé : plus un athlète court rapidement, plus il gagne de points.

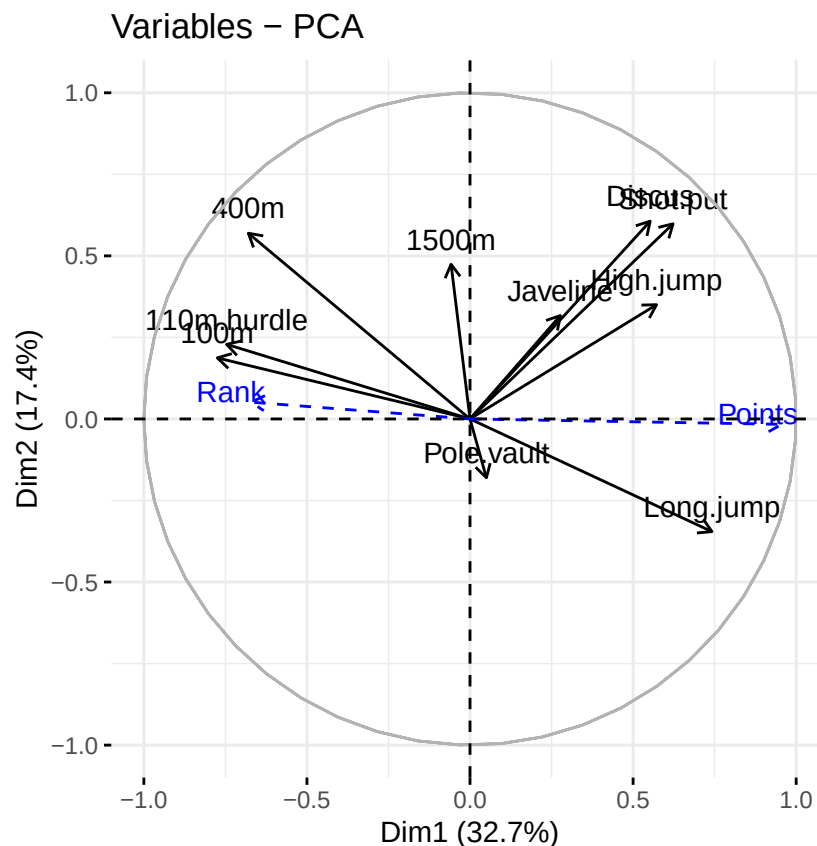
Le premier axe oppose les athlètes qui sont "bons partout" comme Karpov pendant les Jeux Olympiques à ceux qui sont "mauvais partout" comme Bourguignon pendant le Décaster. Cette dimension est particulièrement liée aux variables de vitesse et de saut en longueur qui constituent un groupe homogène. Si on devait donner un nom à cet axe ça serait l'axe "Agilité".

Le deuxième axe oppose les athlètes qui sont forts (variables "Discus" et "Shot.put") à ceux qui ne le sont pas. Si on devait donner un nom à cet axe ça serait l'axe "Force".

Les variables "Discus", "Shot.put" et "High.jump" ne sont pas très corrélées aux variables "X100m", "X400m", "X110m.hurdle" et "Long.jump". Cela signifie que force et vitesse ne sont pas très corrélées.

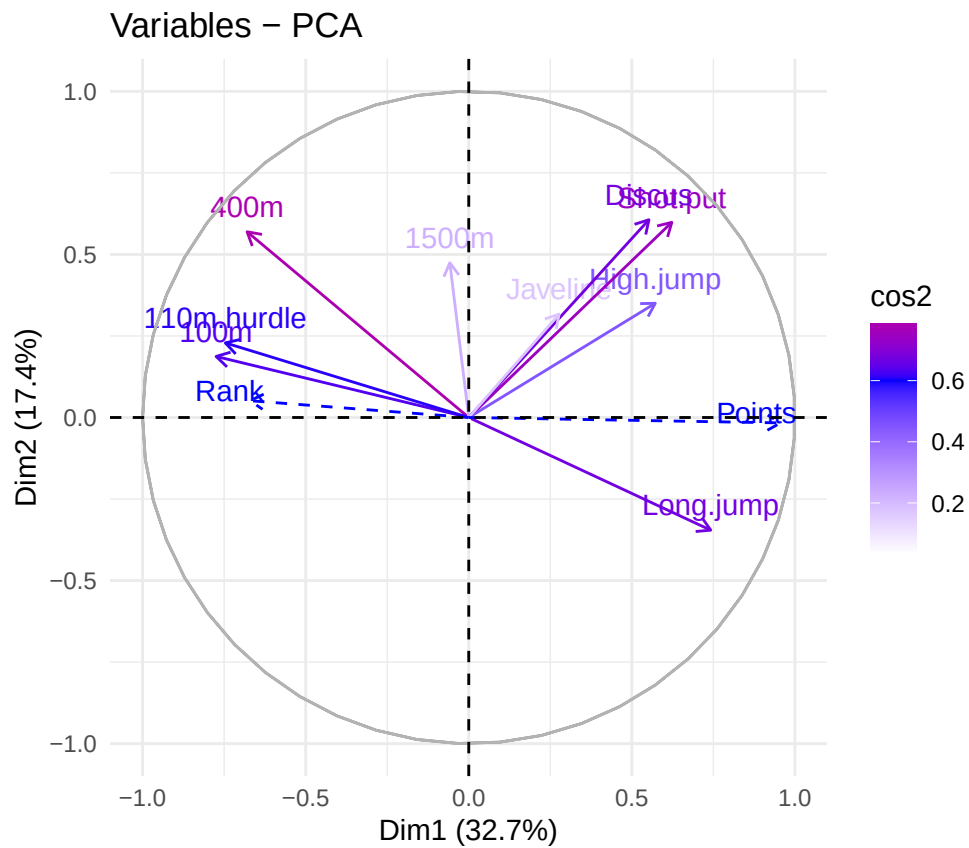
Variable quantitatives supplémentaires. Les variables les plus liées, positivement (resp. négativement), au nombre de points (resp. Rang) sont les variables qui réfèrent à la vitesse ("X100m", "X110m.hurdle", "X400m") et au saut en longueur. Au contraire, "Pole-vault" et "X1500m" n'ont pas une grande influence sur le nombre de points.

```
fviz_pca_var(res.pca)
```



```
fviz_pca_var(res.pca, col.var="cos2") +  
  scale_color_gradient2(low="white", mid="blue",
```

```
high="red", midpoint=0.6) +  
theme_minimal()
```

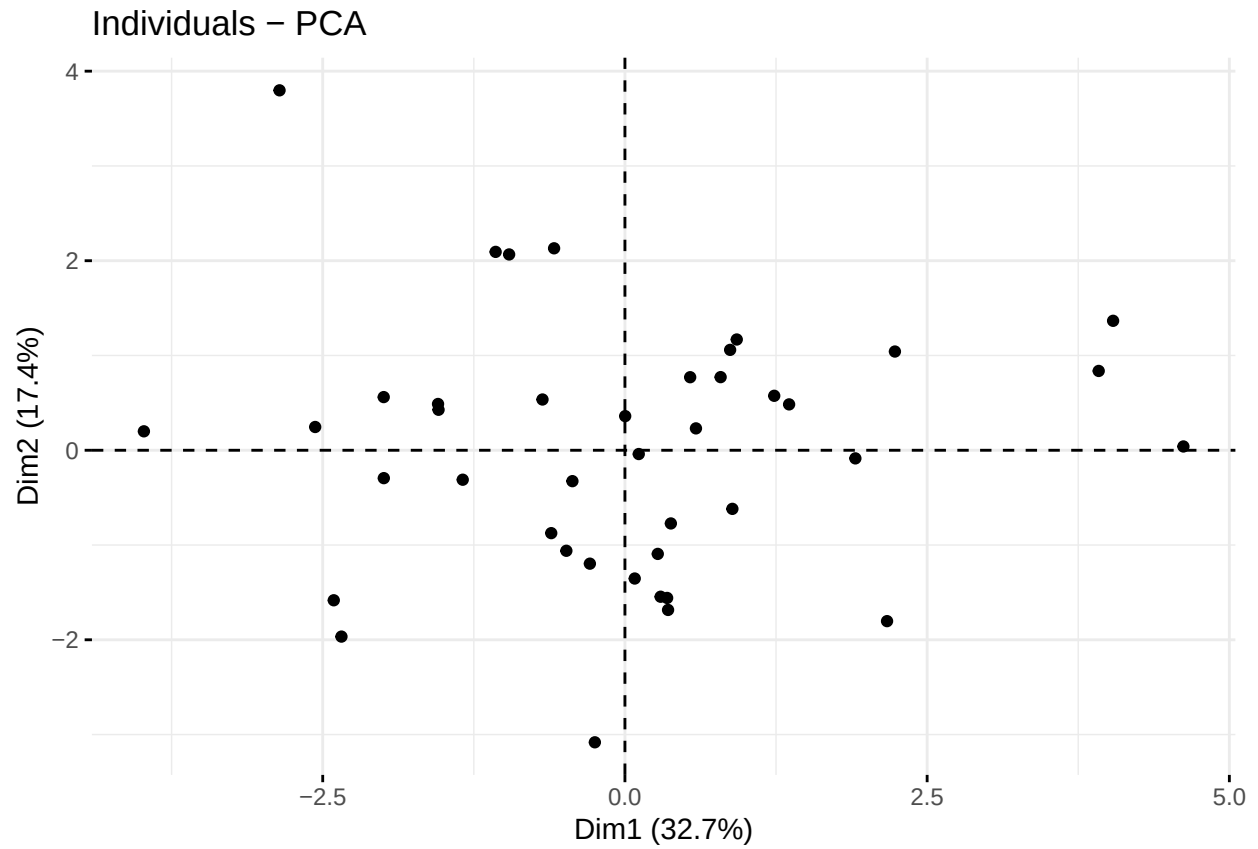


Notons que la qualité de représentation d'une variable sur le premier plan est donnée par la somme de ses cos2 sur chacun des 2 premiers axes.

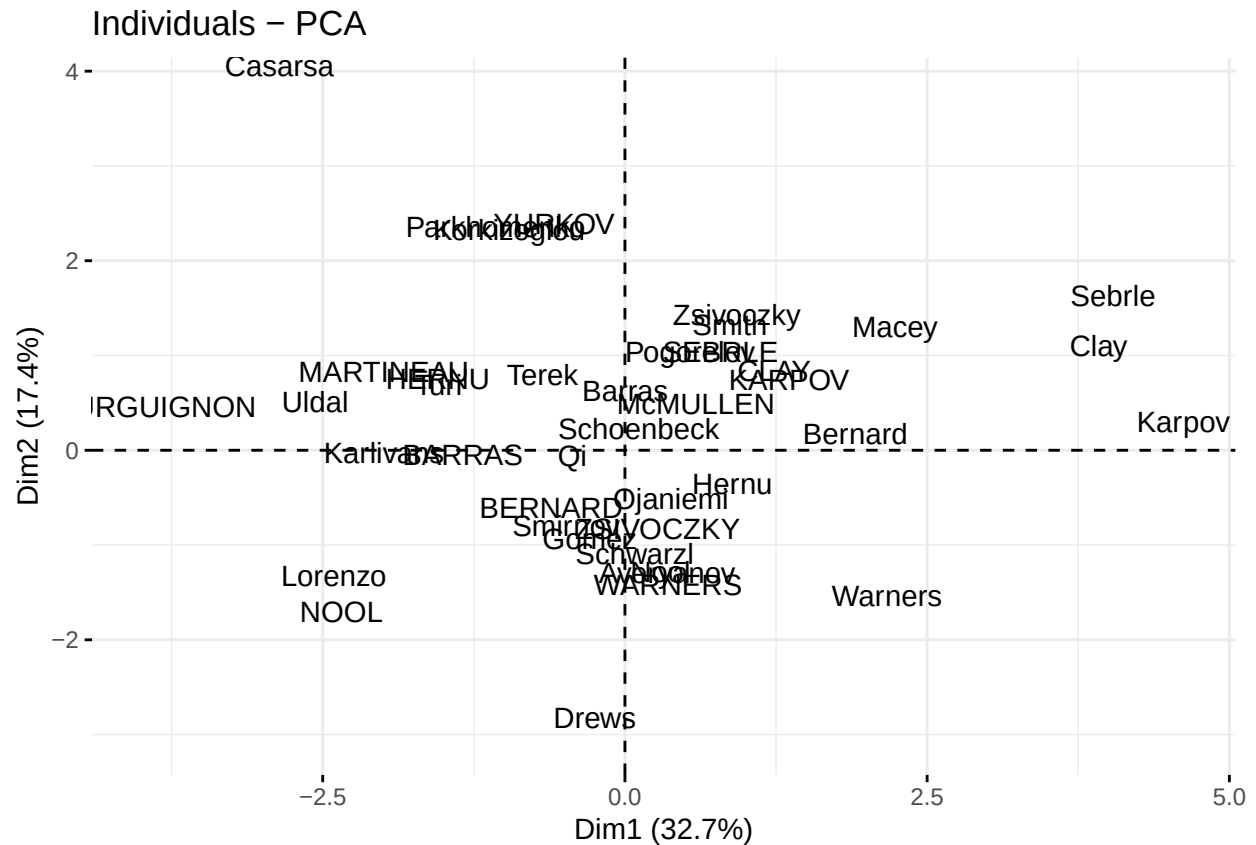
4. Interprétation de la carte des individus

De la même manière, l'objet ind de res.pca contient les objets : coord, cos2 et contrib.

```
fviz_pca_ind(res.pca, geom = "point", col.ind.sup = 'gray')
```

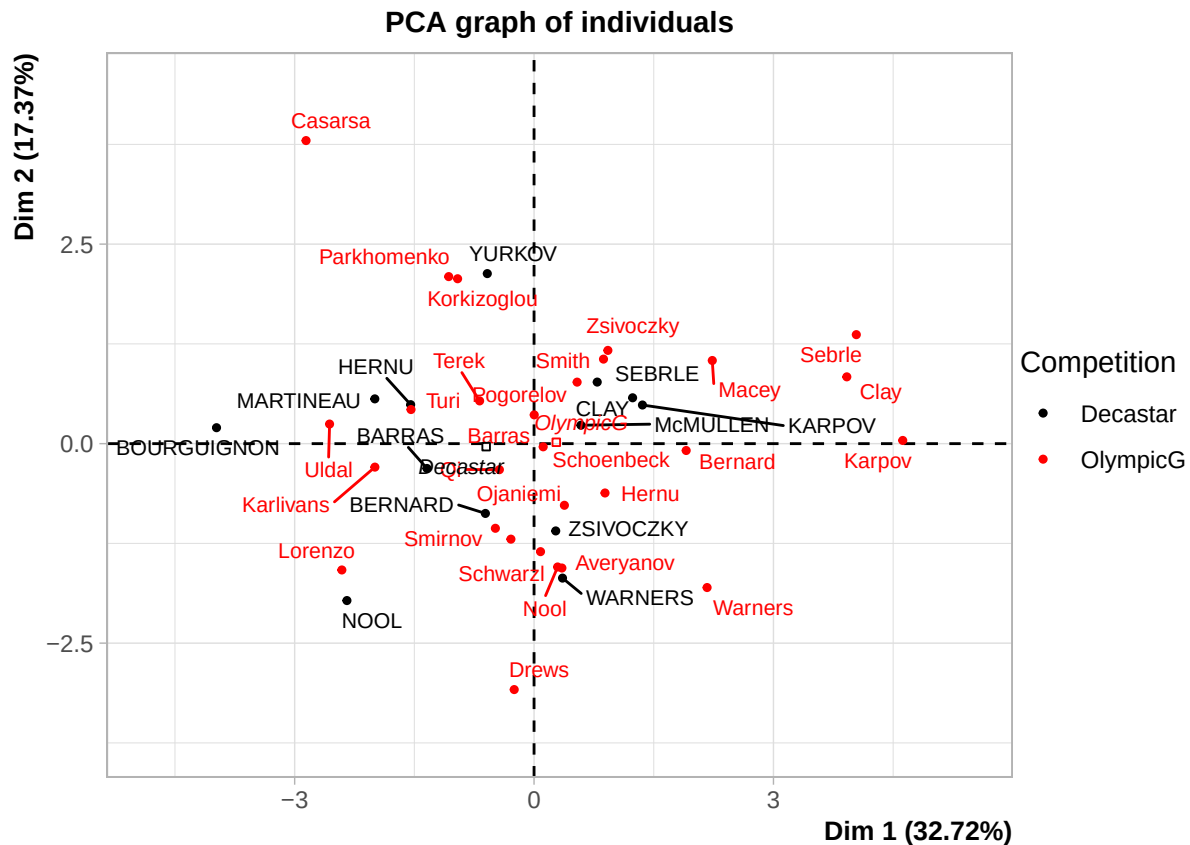


```
fviz_pca_ind(res.pca,geom = "text",col.ind.sup = 'gray')
```



On distingue dans ce premier plan factoriel les groupe suivants : - Les athlètes rapides (comme Karpov) les athlètes rapides et puissants (comme Sebrle), les athlètes lents (comme Bourguignon), les athlètes lents et puissants (comme Casarsa), les athlètes rapides mais faibles (comme Warners) et les ahtlètes ni forts ni rapides (comme Lorenzo).

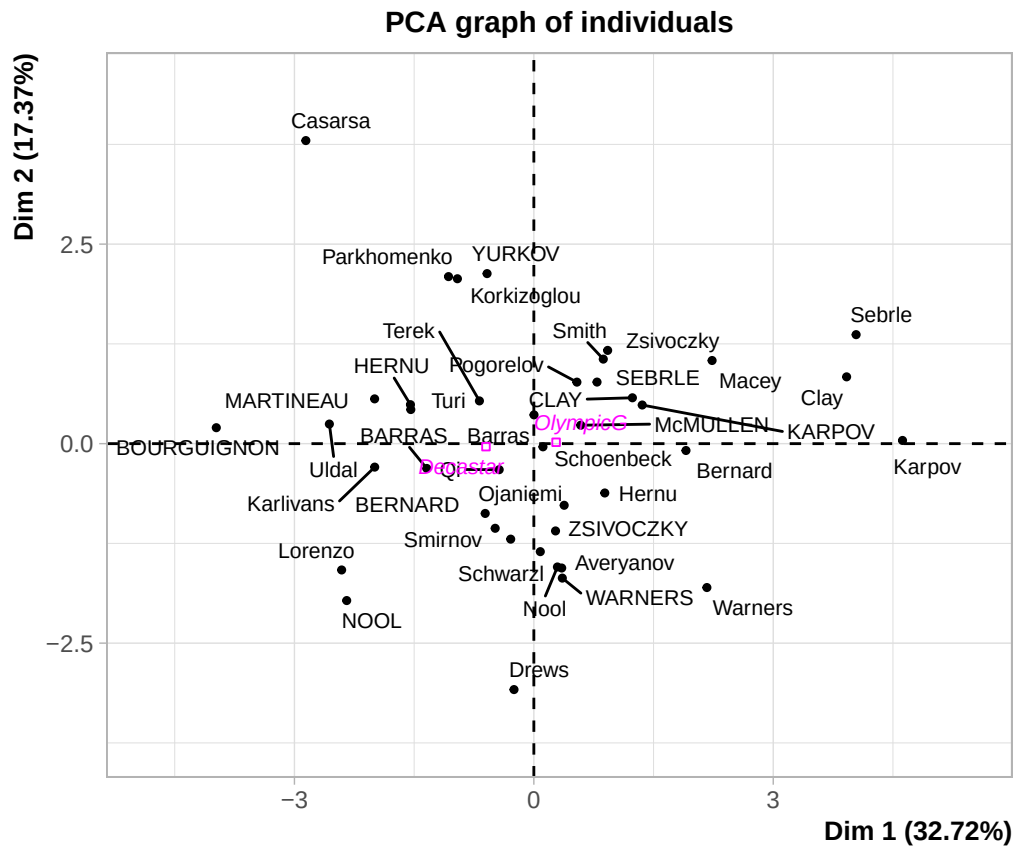
```
fviz_pca_ind(res.pca,geom = "text",col.ind="cos2")+
scale_color_gradient2(low="blue", mid="white",
high="red", midpoint=0.5)
```

En regardant les points qui représentent "Decastar" et "Olympic Games", on voit que "Olympic Games" a une coordonnée plus élevée sur le premier axe que "Decastar". Ceci montre une évolution des performances des athlètes. Tous les athlètes qui ont participé aux deux compétitions ont obtenu des résultats légèrement meilleurs aux jeux Olympiques.

```
res.pca = PCA(decathlon, quanti.sup=c(11: 12), quali.sup=13, graph=F)
plot(res.pca, choix="ind", cex=0.7)
```

```
## Warning: ggrepel: 1 unlabeled data points (too many overlaps). Consider
## increasing max.overlaps
```



Variable qualitative supplémentaire. On peut tirer les mêmes conclusions en nous basant sur les positions des modalités de la variable qualitative supplémentaire Compétition : "Olympic légèrement à droite de l'axe 1 par rapport à la modalité "Decastar".